Java Basics

Identifiers, Keywords, Java Data Types & Operators

Identifiers

- **Identifier** is the **name we give to something in Java** so that Java can identify it.
- In simple words:
  Identifier = Name of a variable, method, class, object, etc.

- Java does not work with meanings; **it works with names**. These names are called identifiers.

Why Identifiers are Needed?

Think like this:

- In real life, we use names to identify people
- In Java, we use identifiers to identify data (Variables) and logic (Methods)

  Without identifiers:
- Java cannot store data
- Java cannot call methods
- Java cannot access classes

  So, identifiers are **mandatory**.

Rules for Identifiers in Java

Rule 1: Allowed characters

An identifier can contain:

- Alphabets (A–Z, a–z)
- Digits (0–9)
- Underscore _
- Dollar sign $

✔ Valid examples:

1. ***age,*** ***2. totalMarks,*** ***3. _count,*** ***4. $salary,*** ***5. student1***

Rule 2: Cannot start with a digit

Identifier **must not start with a number**

❌ Invalid:

1value, 9marks

✔ Valid:

value1, marks9

Rule 3: No spaces allowed

identifiers **cannot contain space**

❌ Invalid:
total marks

✔ Valid:
totalMarks, ***total_marks***

Rule 4: Java keywords cannot be used

Java has **reserved words** (called keywords)
These words already have a meaning in Java

❌ Invalid identifiers:

int, class, public, static, if

** Because Java already uses them.*

Rule 5: No special symbols (except _ and $)

❌ Invalid:

marks%, total#, name@

✔ Valid:

total_marks, $amount

Rule 6: Case-sensitive

Java is **case-sensitive**, which means:

Age, age, and AGE are ***different identifiers***

Naming Conventions (Good Practice – Not Rules)

These are **not compiler rules**, but **industry standards**.

Variable & Method Names

- Start with **lowercase**
- Use **camelCase**

Examples:

totalMarks, calculateSum, studentAge

Class Names

- Start with **uppercase**
- Use **PascalCase**

- Examples:

Student, ***BankAccount,*** ***EmployeeDetails***

Constant Names

- Written in **UPPERCASE**
- Words separated by _

Examples:

MAX_SIZE, ***PI_VALUE***

Keywords

What is a Keyword in Java?

Keyword is a **reserved word** in Java that already has a **fixed meaning**.

Simple definition
Keyword = Special word that Java understands by default
Java uses keywords to:

- define structure
- control program flow
- declare data
- manage memory
- handle exceptions

- So, **we cannot change their meaning**.

Why Keywords Cannot Be Used as Names?

Because:

- Java already uses them for its own work
- If we use them as names, Java will get confused

  Example
- int → already means data type
- class → already means blueprint
- if → already means condition

That's why **keywords cannot be identifiers**.

Important Properties of Java Keywords

Fixed meaning

- Each keyword does **only one job**, always.

Case-sensitive

- All Java keywords are written in **lowercase**
- class is valid keyword
- Class is **NOT** a keyword (but can be an identifier)

Limited in number

- Java has **around 50+ keywords**

Keywords vs Identifiers

Keywords	Identifiers
Reserved words	User-defined names
Fixed meaning	Meaning decided by programmer
Cannot be changed	Can be anything (valid rules)
Cannot be used as names	Used to name variables, methods, classes

Java Keywords Table

abstract	continue	for	new	throws
assert	default	goto*	package	volatile
boolean	do	if	private	transient
break	double	implements	protected	while
byte	else	import	public	try
case	enum	instanceof	return	void
catch	extends	int	short	switch
char	final	interface	static	synchronized
class	finally	long	strictfp	this
const*	float	native	super	throw

Data Types

What is a Data Type?

“Data type = type of value stored in memory.”

Examples in real life:

- Age → number
- Name → text
- Percentage → decimal
- True / False → logical value

Why Java Needs Data Types?

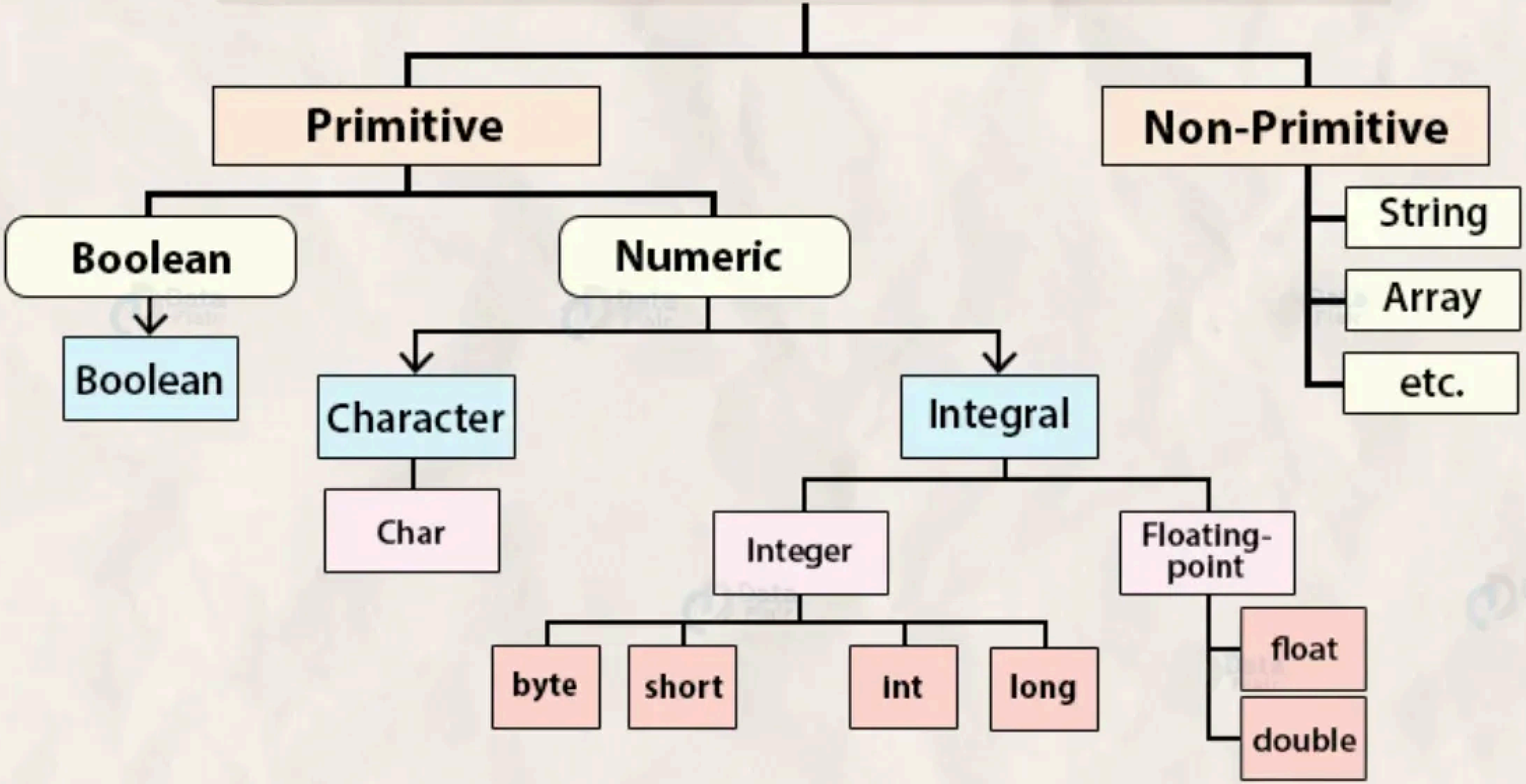
Java is a **strongly typed language**, which means:

- Every variable must have a data type
- Java checks data type at **compile time**

Java uses data types to:

- Allocate correct memory
- Avoid invalid operations
- Improve performance
- Increase program safety

Data Types in Java



Primitive Data Types

Integer Types

Data Type	Size	Range
byte	1 byte (8 bits)	-128 to 127
short	2 bytes (16 bits)	-32,768 to 32,767
int	4 bytes (32 bits)	-2^{31} to $(2^{31} - 1) \approx$ -2.1 billion to +2.1 billion
long	8 bytes (64 bits)	-2^{63} to $(2^{63} - 1)$

Floating-Point Types

Data Type	Size	Range (Approx)
float	4 bytes	**$\pm 3.4 \times 10^{38}$** (up to ~6–7 digits precision)
double	8 bytes	**$\pm 1.7 \times 10^{308}$** (up to ~15–16 digits precision)

Character Type

Data Type	Size	Range (Approx)
float	4 bytes	**$\pm 3.4 \times 10^{38}$** (up to ~6–7 digits precision)
double	8 bytes	**$\pm 1.7 \times 10^{308}$** (up to ~15–16 digits precision)

Boolean Type

Data Type	Size	Range (Approx)
float	4 bytes	**$\pm 3.4 \times 10^{38}$** (up to ~6–7 digits precision)
double	8 bytes	**$\pm 1.7 \times 10^{308}$** (up to ~15–16 digits precision)

Why char range is NOT -128 to 127

Reality:

- char → **1 byte (8 bits)** → signed → **-128 to 127 (C++)**
- char → **2 bytes (16 bits)** → **unsigned (Java)**

Why Java designed char like this?

Java supports **Unicode**, not only English letters

- Unicode includes:
  - English
  - Hindi
  - Punjabi
  - Chinese
  - Symbols, emojis, etc.

Unicode needs **more memory**, so Java uses **16 bits** for char.

Range calculation:

- 16 bits → 2^{16} values = **65,536**
- Since char is **unsigned**: 0 to 65,535

Non-Primitive

Non-primitive data types:

- Store **address/reference**
- Can store **multiple values**
- Created by programmer or Java library

Common Non-Primitive Types

Type	Purpose
String	Store text
Array	Store multiple values
Class	User-defined blueprint
Interface	Abstraction
Object	Real-world entity

Primitive vs Non-Primitive

Primitive	Non-Primitive
Stores value directly	Stores address
Fixed size	Size varies
Faster	Slightly slower
Cannot be null	Can be null
Lower memory	Higher memory

Operators

What is an Operator?

- **Operator** is a **symbol** that performs an **operation** on one or more operands.

  Simple line:
  “Operator = symbol that tells Java what action to perform.”

  Example (conceptual):
- + → addition
- > → comparison
- = → assignment

Types of Operators in Java

Arithmetic Operators

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus (remainder)

% is very important for logic building (even/odd, loops).

Assignment Operators

Operator	Meaning
=	Assign
+=	Add and assign
-=	Subtract and assign
*=	Multiply and assign
/=	Divide and assign

Relational (Comparison) Operators

Operator	Meaning
==	Equal to
!=	Not equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

Logical Operators

Operator	Meaning
&&	Logical AND
||	Logical OR

Explaination:

- AND → all conditions must be true
- OR → any one condition true

Unary Operators

Operator	Meaning
++	Increment
--	Decrement
+	Unary plus
-	Unary minus
!	Logical NOT

Ternary Operator

Shortcut for **if–else**.

Syntax idea:

```
condition ? value_if_true : value_if_false
```

Bitwise Operators (Intro Only)

Used for **bit-level operations**.

Operator	Name
&	Bitwise AND
|	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT
<<	Left Shift
>>	Signed Right Shift
>>>	Unsigned Right Shift

Operator Precedence

Operators	Associativity	Type
++ --	Right to left	Unary postfix
++ -- + - ~ ! (type)	Right to left	Unary prefix
* / %	Left to right	Multiplicative
+ -	Left to right	Additive
<< >> >>>	Left to right	Shift
< <= > >=	Left to right	Relational
== !=	Left to right	Equality
&	Left to right	Boolean Logical AND
^	Left to right	Boolean Logical Exclusive OR
|	Left to right	Boolean Logical Inclusive OR
&&	Left to right	Conditional AND
||	Left to right	Conditional OR
?:	Right to left	Conditional
= += -= *= /= %=	Right to left	Assignment